

GLA UNIVERSITY, GREATER NOIDA

Greater Noida – 201310, Uttar Pradesh, India



FraudShield Real-Time Credit Card Fraud Detection Gateway

SOFTWARE REQUIREMENT SPECIFICATION

Version 1.0

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE & ENGINEERING - AI

Submitted By

Ayush Sharma

Roll No.:9 | Section: ND

Under the Supervision of

Mr. Palash Raghuwanshi

GLA University, Greater Noida

Reviewed By

Mr. Palash Raghuwanshi

GLA University, Greater Noida

SCHOOL / DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING / APPLICATIONS

GLA UNIVERSITY, GREATER NOIDA – 201310

September 2026

Revision History

Version	Date	Change	Reason
1.0	04-09-2026	Initial version	First submission

1. Introduction

1.1 Purpose

This document specifies the requirements of FraudShield v1.0, a real-time credit card fraud detection gateway for a card-issuing bank's Customer, Merchant, Fraud Analyst, and Admin users. It is the baseline against which the implementation and its test cases are verified.

1.2 Scope

FraudShield scores simulated transactions in real time, routes them to auto-approve, auto-decline, or a Fraud Analyst review queue, and lets an Admin manage the detection threshold and user accounts. The system does not connect to a real payment network, does not move real money, and does not retrain its classifier automatically.

Term	Definition
Fraud score	The classifier's output for a transaction: a number between 0 and 1 representing estimated fraud probability.
Review queue	The ordered list of transactions with status "under review", visible only to Fraud Analysts.
Rule-only fallback	The transaction status produced when the trained classifier cannot return a score, and only the rule pre-filter has run.
Threshold	Admin-configured fraud-score boundaries separating auto-approve, review, and auto-decline.

2. Overall Description

2.1 User Classes

User class	Who they are	What they may do
Customer	Authenticated end user submitting purchases	Submit a transaction; view its own status
Merchant	Authenticated business account	View transactions made at their business and each one's status
Fraud Analyst	Staff reviewer	View the review queue; approve or decline a queued transaction
Admin	System owner	Set the detection threshold; manage user accounts

2.2 Assumptions and Dependencies

- The system depends on Vercel's free-tier serverless functions, which cold-start after a period of inactivity; the first request after idle is slower than the rest and is not something the system controls.
- The system depends on Neon's free-tier PostgreSQL using a pooled connection string; a direct (non-pooled) connection would exhaust available connections under serverless load.
- The system depends on the IEEE-CIS and Sparkov-simulated datasets remaining downloadable from Kaggle at their current URLs for the duration of the project.
- The system depends on Twilio, or SMTP email if Twilio is unavailable, for analyst notifications; no notification is sent if both are unreachable.

2.3 Constraints

C1: The classifier may only use techniques covered in the course ML syllabus — no architecture beyond that list, regardless of whether a more advanced one would score higher.

C2: The two source datasets have incompatible feature schemas (anonymized numeric columns vs. readable categorical/location columns), so the classifier cannot be trained as a single model across both.

C3: The system must run within Vercel's and Neon's free-tier resource and time limits, since no paid infrastructure is budgeted for this project.

3. Data Source and Baseline

Item	Value	Required now?
Dataset name	IEEE-CIS Fraud Detection	Yes
Source and URL	Kaggle competition — kaggle.com/c/ieee-fraud-detection	Yes
Licence / terms of use	Kaggle competition data, for competition and non-commercial research/educational use under Kaggle's terms; a free Kaggle account is required to download	Yes
Number of rows and columns	590,540 rows (train_transaction.csv) joined with 144,233 rows (train_identity.csv) on TransactionID; 434 columns after the join	Yes
Prediction target	isFraud (binary: 1 = fraud, 0 = legitimate)	Yes
Task type	Predicting one of two categories (binary classification)	Yes
Class balance	20,663 fraud rows out of 590,540 (3.5%); 96.5% legitimate	Yes
Primary evaluation metric	Recall	Yes
Why that metric	A Fraud Analyst reviews anything flagged, so a false alarm costs a few minutes of review; fraud that's never flagged is a loss the system had one chance to catch — missed fraud costs more than false alarms	Yes
Level 0 baseline and its score	Always predict "not fraud": 0% recall by definition, 96.5% accuracy — exactly why accuracy isn't used here	Yes
Known data quality problems	Substantial missingness in the identity columns (some >90% missing), since only ~24% of transactions have a matching identity record; several V-columns are also partially missing	Yes
Level 1 baseline and its score	<To be measured in Sprint I — logistic regression on the readable columns only.>	Later
Target for this release	<To be set once the Level 1 score is known — log it in Revision History.>	Later

4. System Architecture

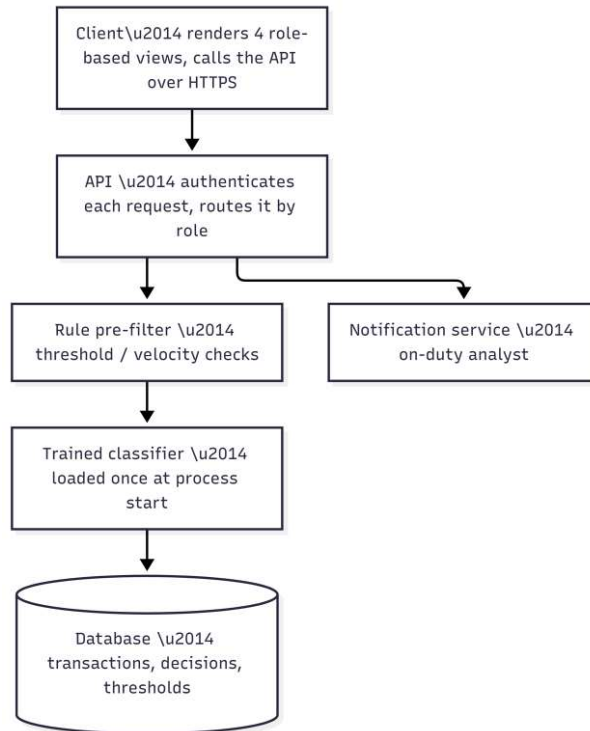


Figure 1: FraudShield component architecture and data flow.

Component	Responsibility
Client	Renders the four role-based views and calls the API over HTTPS.
API	Authenticates each request and routes it to the right handler by role.
Rule pre-filter	Applies fixed threshold/velocity checks before the classifier runs.
Trained classifier	Loaded once at process start; returns a fraud-probability score for an already-validated request. Training happens offline, in a separate script — not in this component.
Database	Persists transactions, analyst decisions, and threshold history.
Notification service	Sends an alert to the on-duty analyst for urgent flags.

5. Functional Requirements

5.1 Transaction Submission & Validation

FR-1.1: The system shall allow an authenticated Customer to submit a transaction specifying a merchant, an amount, and a timestamp.

FR-1.2: The system shall reject a submitted transaction whose amount is not a positive number, naming the amount of field in the error.

FR-1.3: The system shall reject a submitted transaction that does not specify a registered merchant, naming the merchant field in the error.

FR-1.4: The system shall display the resulting status — approved, declined, or under review — on the same screen the Customer used to submit the transaction, without a page to reload.

5.2 Model-backed Requirements

FR-2.1: The system shall accept a validated transaction as classifier input and reject one missing a required feature column, naming the missing column in the error.

FR-2.2: The system shall return a fraud-probability score between 0 and 1 for a valid classifier request.

FR-2.3: The system shall present the fraud-probability score to the Fraud Analyst together with which layer — rule or model — produced the flag.

FR-2.4: The system shall, when the trained classifier is unavailable or returns no usable score, route the transaction using the rule pre-filter result alone, recording its status as rule-only.

FR-2.5: The system shall allow a Fraud Analyst to change the system's routing decision on any transaction still in the review queue.

5.3 Review Queue & Analyst Actions

FR-3.1: The system shall list every transaction with status "under review" in the Fraud Analyst's queue, ordered by descending fraud score.

FR-3.2: The system shall record an approve-or-decline decision against the transaction it was made on, together with the analyst's identifier and a timestamp.

FR-3.3: The system shall reject a decision submitted for a transaction that already has a recorded decision, naming the existing decision in the error.

FR-3.4: The system shall send a notification to an on-duty Fraud Analyst within 60 seconds of a transaction's fraud score exceeding the configured urgency level.

5.4 Admin & Threshold Management

FR-4.1: The system shall allow an Admin to set the fraud-score values that separate auto-approve, review, and auto-decline ranges.

FR-4.2: The system shall reject a threshold change submitted without an accompanying rationale, requiring one before saving.

FR-4.3: The system shall store every threshold change as a new record rather than overwriting the previous value.

FR-4.4: The system shall revoke a deactivated user account's access on that account's next request, without requiring the user to log out first.

6. Non-Functional Requirements

NFR-1: The system shall return a fraud-probability score for a submitted transaction within 5 seconds at the 95th percentile, excluding the first request after 15 minutes of platform inactivity.

NFR-2: The system shall require a valid JWT bearer token, checked server-side against the requesting user's role, on every endpoint other than login and registration.

NFR-3: The system shall record every scored transaction's model-version identifier, input feature snapshot, and output score, retrievable by transaction ID.

NFR-4: The system shall refuse to start the API process if the loaded model file's scikit-learn version does not match the installed scikit-learn version, logging both version numbers.

7. Build Plan

7.1 Minimum Viable Product

FR / NFR ID	Requirement (short)	Why this one is in the MVP
FR-1.1	Submit a transaction	Without intake, nothing downstream has data to act on.
FR-1.2	Reject invalid amount	Later stages can't be tested against realistic, bad input until validation exists.
FR-2.4	Rule-only fallback routing	The trained classifier won't exist yet in Sprint I; this is what makes the system usable before FR-2.1–2.3 are built.
FR-3.1	Analyst review queue	The second half of the loop FR-2.4's flagged transactions need to go somewhere.
FR-3.2	Record analyst decision	A decision that isn't recorded can't become labelled data for later model work.
NFR-2	JWT auth on every endpoint	Every other requirement sits behind role-based access; retrofitting auth later touches every endpoint twice.
NFR-3	Record score + model version	Later evaluation depends on data captured from the first scored transaction on, not backfilled.

Deliberately not in the MVP: FR-2.2 (returns a model-based fraud probability) — needs a trained, evaluated classifier first, which depends on the baseline and metric work in Section 3 being finished.

7.2 Build Order

For reference, the fixed project stages:

Stage	Deliverable	Date
Development Sprint I	Prototype / initial working model	27 Sep 2026
Development Sprint II	50% functional application	17 Oct 2026
Development Sprint III	75–80% functional application	7 Nov 2026
Testing & Validation	Test report and bug log	17 Nov 2026
Deployment & Documentation	Deployed application, user manual, technical documentation	30 Nov 2026
Final Presentation	Final report, presentation and viva	5–10 Dec 2026

Your plan:

Stage	What will be working by then	FR / NFR IDs	How I will know it is done
Sprint I: 27 Sep	MVP scaffold: submission, validation, rule-only routing, analyst queue and decisions	FR-1.1, FR-1.2, FR-2.4, FR-3.1, FR-3.2, NFR-2, NFR-3	A submitted transaction reaches the queue and can be approved or declined, entirely without a trained model.
Sprint II: 17 Oct	Full intake, plus the trained classifier live in the pipeline	FR-1.3, FR-1.4, FR-2.1, FR-2.2, FR-2.3, FR-3.3	The same demo as Sprint I now shows a model-based score, not just the rule-only fallback.
Sprint III: 7 Nov	Analyst override, urgency notifications, full admin control	FR-2.5, FR-3.4, FR-4.1, FR-4.2, FR-4.3, FR-4.4	An Admin changes the threshold with a logged reason; an urgent flag reaches an analyst without polling.
Testing: 17 Nov	All of the above, verified — nothing new built	AC-1 to AC-4 (Section 10)	Every acceptance criterion passes in a fresh run, including the failure path in AC-1.
Deployment: 30 Nov	Live URL; NFR-1 and NFR-4 verified under real deployment	NFR-1, NFR-4	A cold request after 15 min idle still meets NFR-1; a deliberately mismatched model file is refused per NFR-4.

8. Data Requirements

8.1 Entities

Entity: Transaction — created by FR-1.1, updated by FR-2.4 and FR-3.2.

Field	Type	Constraint / Notes
id	UUID	Primary key.
customer_id	UUID (FK→User)	Not null
merchant_id	UUID (FK→MerchantProfile)	Not null
amount	Decimal	Must be greater than 0
submitted_at	Timestamp	Not null; must not be a future date
fraud_score	Float	Nullable until scored; range 0 to 1
status	Enum	pending / approved / declined / under_review; default pending

Other entities: User, MerchantProfile, AnalystDecision, ThresholdConfig.

8.2 Entity Relationship Diagram

[REMOVE] What goes here: One ER diagram covering every entity in 8.1, with cardinalities marked on every relationship.

Done when: Every entity in 8.1 appears, and every line is labelled 1:1, 1:N or M:N.

9. Design Decisions and Alternatives Considered

#	Decision	Alternative rejected	Reason
D 1	Train two separate classifiers, one per dataset, instead of merging them.	A single model on a merged/joint table.	IEEE-CIS's anonymized V-columns have no equivalent in the Sparkov dataset's readable columns; a common schema would discard most of one dataset.
D 2	Serve the model in-process inside the Django API, loaded via joblib at process start.	A standalone inference microservice over HTTP.	One classifier, no need to scale inference independently of API traffic — a second service adds a deployment to keep in sync for no measured benefit.
D 3	Apply SMOTE after the train/test split, inside the training pipeline only.	Oversampling the full dataset before splitting.	Oversampling before the split duplicates minority rows across both sides, so a near-identical row can appear in train and test, inflating recall in a way that won't hold up later.

10. Acceptance Criteria

AC-1: Submitting a transaction with amount = −5 through the Customer form produces an on-screen error naming the amount field, and no transaction record is created.

AC-2: Submitting a valid transaction with the trained classifier intentionally stopped produces a status of "under review" via the rule-only fallback, not an unhandled error.

AC-3: A transaction routed to "under review" appears in the Fraud Analyst's queue on the same page load, sorted above any lower-scored transaction already there.

AC-4: An Analyst approving a queued transaction removes it from the queue and records the analyst's identifier and a timestamp, retrievable from the transaction detail view.

11. Out of Scope for This Release

Real payment-gateway integration: The system scores simulated transactions only; connecting to an actual card network needs PCI-DSS-scoped infrastructure this project does not build.

Continuous / online retraining: Analyst decisions are logged as labelled data (FR-3.2), but nothing in this release automatically retrains the classifier on it — retraining stays a manual, offline step.

SHAP-based per-prediction explanations in the Fraud Analyst UI: This is the reason the Sparkov dataset's readable columns were chosen, but the visualization itself didn't fit the 12-week window — wanted and cut for time.

Appendix A: State Model

[REMOVE] **What goes here:** *Applies only if something in your system moves through named states — a job, an upload, a training run, a request.*

Done when: *Every state is reachable from the initial state, every terminal state is marked terminal, and no state can be entered and never left.*

Appendix B: Traceability

[REMOVE] **What goes here:** *One row per requirement, linking it to the task card that implements it and the test case that verifies it. Start with your MVP requirements from 7.1.*

Done when: *Every MVP requirement has a task card ID. A requirement with no card means either it is not planned or the plan is incomplete — both are worth knowing.*

FR ID	Requirement (short)	In MVP?	Task card ID	Test case ID
FR-1.1	<short description>	<Yes / No>	<board id>	<later>
FR-1.2	<short description>	<Yes / No>	<board id>	
<...>				